

LAB ASSIGNMENT – 11.5

NAME – Anushka Boora

Ht No.: 2303A510E6

BATCH – 29

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

```
class Stack:
```

```
    pass
```

Expected Output:

- A functional stack implementation with all required methods and docstrings.

Code:

```
# Generate a simple Python class for Stack with push, pop, peek, and is_empty methods including basic docstrings
```

```
class Stack:
```

```
    """Simple Stack implementation using a list."""
```

```
    def __init__(self):
```

```
        """Initialize an empty stack."""
```

```
        self.items = []
```

```
    def push(self, item):
```

```
        """Add an item to the top of the stack."""
```

```
        self.items.append(item)
```

```
def pop(self):  
    """Remove and return the top item of the stack."""  
    if not self.is_empty():  
        return self.items.pop()  
    return "Stack is empty"
```

```
def peek(self):  
    """Return the top item without removing it."""  
    if not self.is_empty():  
        return self.items[-1]  
    return "Stack is empty"
```

```
def is_empty(self):  
    """Check if the stack is empty."""  
    return len(self.items) == 0
```

Example usage:

```
if __name__ == "__main__":  
    stack = Stack()  
    stack.push(1)  
    stack.push(2)  
    print(stack.peek()) # Output: 2  
    print(stack.pop()) # Output: 2  
    print(stack.is_empty()) # Output: False  
    print(stack.pop()) # Output: 1  
    print(stack.is_empty()) # Output: True
```

Output:

```

1 # Generate a simple Python class for Stack with push, pop, peek, and is_empty methods including basic docstrings
2 class Stack:
3     """Simple Stack implementation using a list."""
4
5     def __init__(self):
6         """Initialize an empty stack."""
7         self.items = []
8
9     def push(self, item):
10        """Add an item to the top of the stack."""
11        self.items.append(item)
12
13    def pop(self):
14        """Remove and return the top item of the stack."""
15        if not self.is_empty():
16            return self.items.pop()
17        return "Stack is empty"
18
19    def peek(self):
20        """Return the top item without removing it."""
21        if not self.is_empty():
22            return self.items[-1]
23        return "Stack is empty"
24
25    def is_empty(self):
26        """Check if the stack is empty."""
27        return len(self.items) == 0
28
29 # Example usage:
30 if __name__ == "__main__":
31     stack = Stack()
32     stack.push(1)
33     stack.push(2)
34     print(stack.peek()) # Output: 2
35     print(stack.pop())  # Output: 2
36     print(stack.is_empty()) # Output: False
37
38 ncher' '61205' '-' 'C:\Users\boora\Downloads\3-2\AI-AC\acc11.5.py'
39 2
40 2
41 False
42 1
43 True
44 PS C:\Users\boora\Downloads\3-2\AI-AC>

```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

```
    pass
```

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Code:

Generate a simple Python Queue class using lists with enqueue, dequeue, peek, and size methods including basic docstrings.

```
class Queue:
```

```
    """Simple Queue implementation using a list."""
```

```
def __init__(self):
    """Initialize an empty queue."""
    self.items = []

def enqueue(self, item):
    """Add an element to the queue."""
    self.items.append(item)

def dequeue(self):
    """Remove and return the first element from the queue."""
    if not self.is_empty():
        return self.items.pop(0)
    return "Queue is empty"

def peek(self):
    """Return the front element without removing it."""
    if not self.is_empty():
        return self.items[0]
    return "Queue is empty"

def size(self):
    """Return the number of elements in the queue."""
    return len(self.items)

def is_empty(self):
    """Check if the queue is empty."""
    return len(self.items) == 0
```

Example usage:

```

if __name__ == "__main__":

    queue = Queue()

    queue.enqueue(1)

    queue.enqueue(2)

    print(queue.peek()) # Output: 1

    print(queue.dequeue()) # Output: 1

    print(queue.size()) # Output: 1

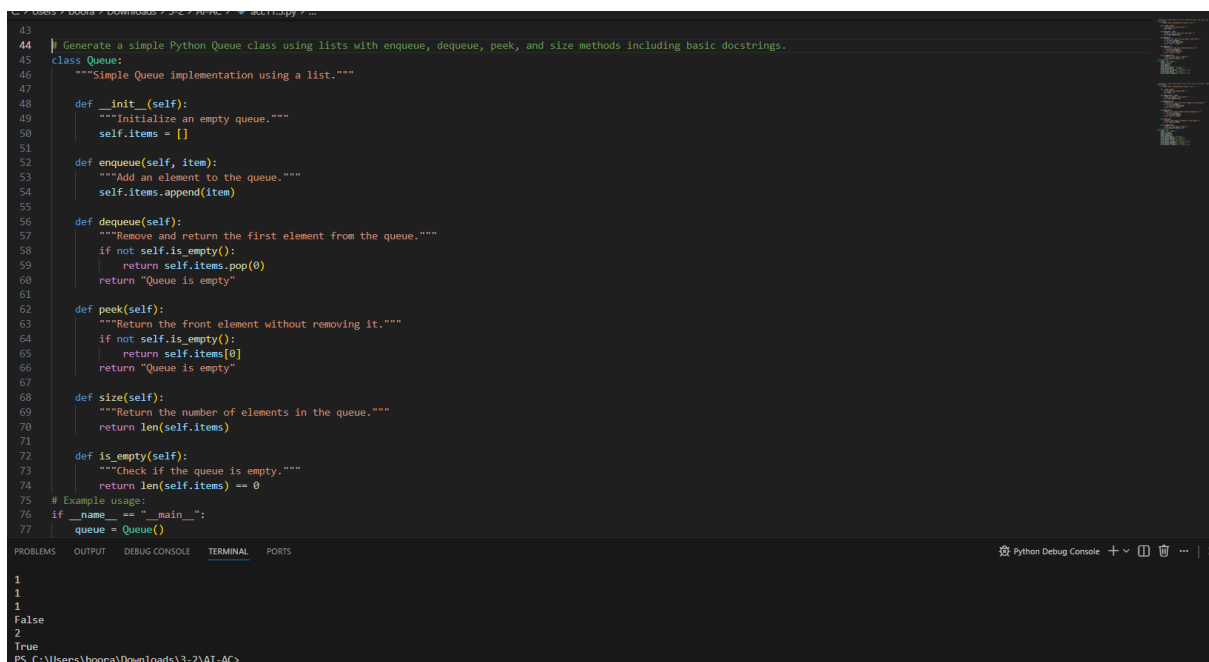
    print(queue.is_empty()) # Output: False

    print(queue.dequeue()) # Output: 2

    print(queue.is_empty()) # Output: True

```

Output:



The screenshot shows a Python IDE with a dark theme. The main editor displays the implementation of a Queue class using a list. The code includes methods for enqueue, dequeue, peek, size, and is_empty. A main block at the bottom demonstrates the usage of the Queue class. The output window at the bottom shows the results of the execution: 1, 1, 1, False, 2, and True.

```

43
44 # Generate a simple Python Queue class using lists with enqueue, dequeue, peek, and size methods including basic docstrings.
45 class Queue:
46     """Simple Queue implementation using a list."""
47
48     def __init__(self):
49         """Initialize an empty queue."""
50         self.items = []
51
52     def enqueue(self, item):
53         """Add an element to the queue."""
54         self.items.append(item)
55
56     def dequeue(self):
57         """Remove and return the first element from the queue."""
58         if not self.is_empty():
59             return self.items.pop(0)
60         return "Queue is empty"
61
62     def peek(self):
63         """Return the front element without removing it."""
64         if not self.is_empty():
65             return self.items[0]
66         return "Queue is empty"
67
68     def size(self):
69         """Return the number of elements in the queue."""
70         return len(self.items)
71
72     def is_empty(self):
73         """Check if the queue is empty."""
74         return len(self.items) == 0
75
76 # Example usage:
77 if __name__ == "__main__":
78     queue = Queue()
79
80     queue.enqueue(1)
81     queue.enqueue(2)
82     print(queue.peek())
83     print(queue.dequeue())
84     print(queue.size())
85     print(queue.is_empty())
86     print(queue.dequeue())
87     print(queue.is_empty())

```

1
1
1
False
2
True
PS C:\Users\boora\Downloads\3-2\AT-AC>

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

```
class Node:
```

```
    pass
```

```
class LinkedList:
```

```
    pass
```

Expected Output:

- A working linked list implementation with clear method documentation.

Code:

Generate a simple Python implementation of a Singly Linked List with Node and LinkedList classes including insert and display methods with basic docstrings.

```
class Node:
```

```
    """Node class for storing data and next reference."""
```

```
    def __init__(self, data):
```

```
        """Initialize node with data."""
```

```
        self.data = data
```

```
        self.next = None
```

```
class LinkedList:
```

```
    """Singly Linked List implementation."""
```

```
    def __init__(self):
```

```
        """Initialize empty linked list."""
```

```
        self.head = None
```

```
    def insert(self, data):
```

```

        """Insert a new node at the end of the list."""
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        temp = self.head
        while temp.next:
            temp = temp.next

        temp.next = new_node

    def display(self):
        """Display all elements in the linked list."""
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

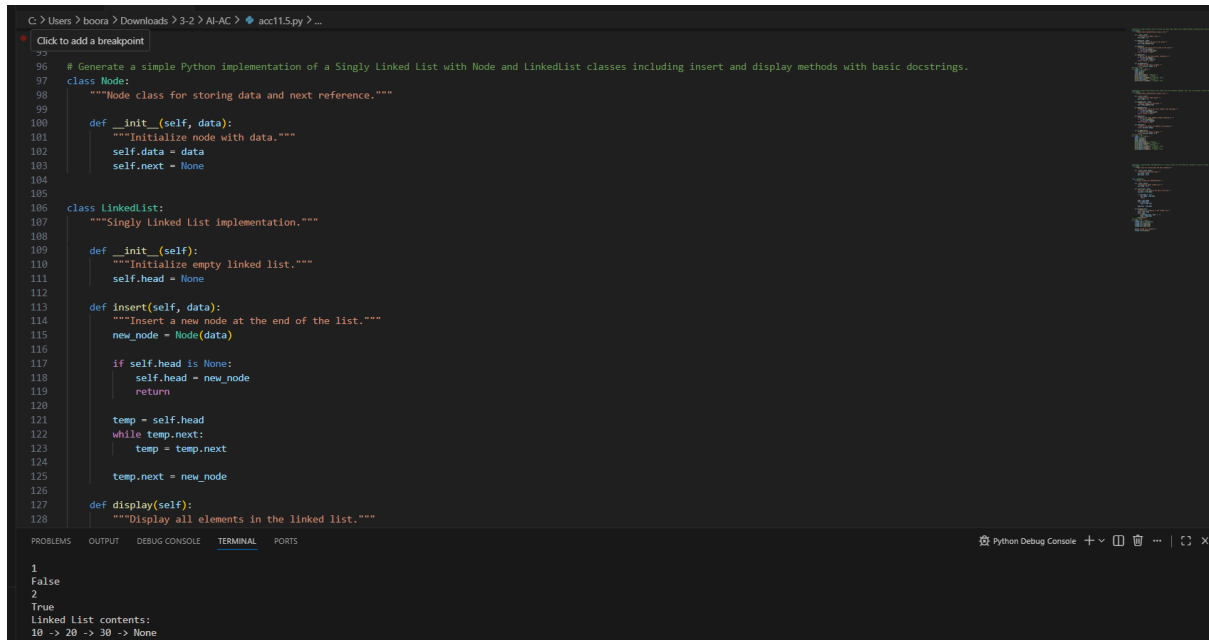
# Example usage
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.insert(10)
    linked_list.insert(20)
    linked_list.insert(30)

    print("Linked List contents:")

```

```
linked_list.display()
```

Output:



The screenshot shows a Python IDE with a dark theme. The main editor displays a Python script for a Singly Linked List. The script includes a `Node` class with `__init__` and `display` methods, and a `LinkedList` class with `__init__`, `insert`, and `display` methods. The `display` method of `LinkedList` prints the contents of the list. The output window at the bottom shows the following text:

```
1
False
2
True
Linked List contents:
10 -> 20 -> 30 -> None
```

Task Description #4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable:
```

```
pass
```

Expected Output:

- Collision handling using chaining, with well-commented methods.

Code:

```
# Generate a simple Python HashTable class using chaining for collision handling with insert, search, and delete methods and basic docstrings.
```

```
class HashTable:
```

```
    """Simple Hash Table implementation using chaining."""
```



```

def __init__(self, size=10):
    """Initialize hash table with given size."""
    self.size = size
    self.table = [[] for _ in range(size)]

def hash_function(self, key):
    """Compute index for a key."""
    return hash(key) % self.size

def insert(self, key, value):
    """Insert key-value pair into the hash table."""
    index = self.hash_function(key)
    self.table[index].append((key, value))

def search(self, key):
    """Search for a key and return its value."""
    index = self.hash_function(key)
    for k, v in self.table[index]:
        if k == key:
            return v
    return "Key not found"

def delete(self, key):
    """Delete a key-value pair from the hash table."""
    index = self.hash_function(key)
    for i, (k, v) in enumerate(self.table[index]):
        if k == key:

```

```
del self.table[index][i]
```

```
return "Deleted"
```

```
return "Key not found"
```

Example usage

```
if __name__ == "__main__":
```

```
    ht = HashTable()
```

```
    ht.insert("name", "Faaaaaaaaaah")
```

```
    ht.insert("age", 30)
```

```
    print(ht.search("name")) # Output: Alice
```

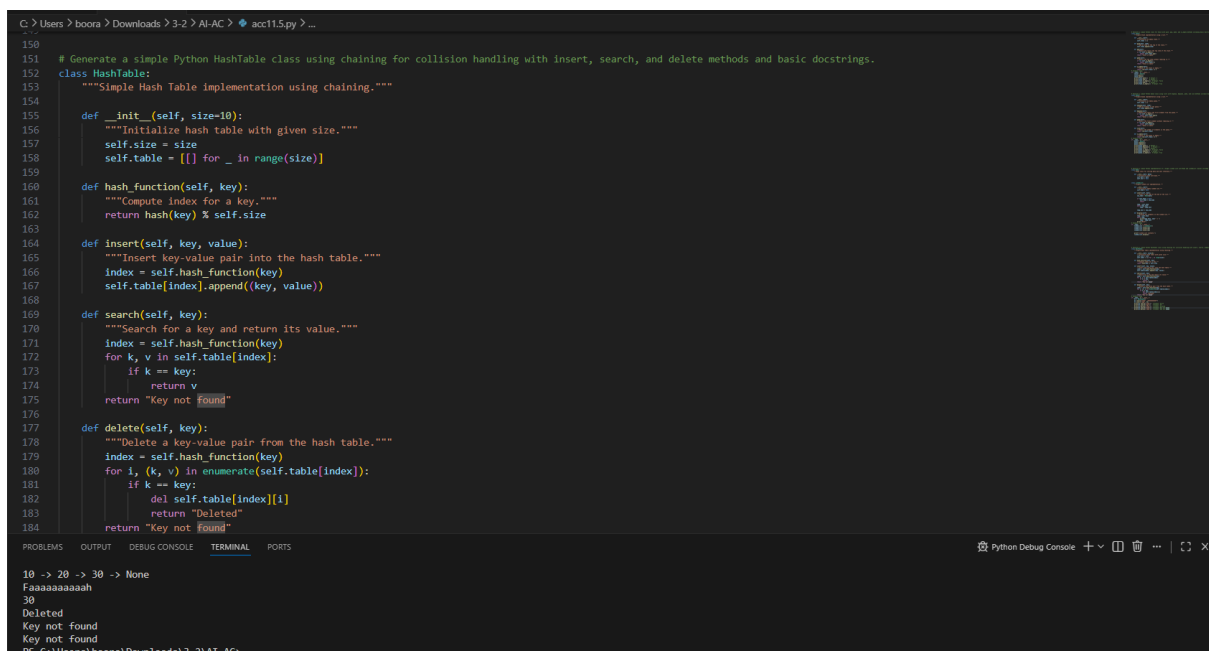
```
    print(ht.search("age")) # Output: 30
```

```
    print(ht.delete("name")) # Output: Deleted
```

```
    print(ht.search("name")) # Output: Key not found
```

```
    print(ht.delete("name")) # Output: Key not found
```

Output:



The screenshot shows a Python IDE with a file named `acc115.py`. The code defines a `HashTable` class using chaining for collision handling. The class has methods for `__init__`, `hash_function`, `insert`, `search`, and `delete`. The `search` method returns the value for a given key, or "Key not found" if the key is not present. The `delete` method removes a key-value pair from the hash table, returning "Deleted" or "Key not found".

```
150 # Generate a simple Python HashTable class using chaining for collision handling with insert, search, and delete methods and basic docstrings.
151 class HashTable:
152     """Simple Hash Table implementation using chaining."""
153
154     def __init__(self, size=10):
155         """Initialize hash table with given size."""
156         self.size = size
157         self.table = [[] for _ in range(size)]
158
159     def hash_function(self, key):
160         """Compute index for a key."""
161         return hash(key) % self.size
162
163     def insert(self, key, value):
164         """Insert key-value pair into the hash table."""
165         index = self.hash_function(key)
166         self.table[index].append((key, value))
167
168     def search(self, key):
169         """Search for a key and return its value."""
170         index = self.hash_function(key)
171         for k, v in self.table[index]:
172             if k == key:
173                 return v
174         return "Key not found"
175
176     def delete(self, key):
177         """Delete a key-value pair from the hash table."""
178         index = self.hash_function(key)
179         for i, (k, v) in enumerate(self.table[index]):
180             if k == key:
181                 del self.table[index][i]
182                 return "Deleted"
183         return "Key not found"
```

The output of the program is shown in the terminal:

```
10 -> 20 -> 30 -> None
Faaaaaaaaaah
30
Deleted
Key not found
Key not found
```

Task Description #5 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

```
class Graph:
```

```
    pass
```

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

Code:

```
# Generate a simple Python Graph class using an adjacency list with methods to add vertices, add edges, and display the graph with basic docstrings.
```

```
class Graph:
```

```
    """Simple Graph implementation using adjacency list."""
```

```
    def __init__(self):
```

```
        """Initialize empty graph."""
```

```
        self.graph = {}
```

```
    def add_vertex(self, vertex):
```

```
        """Add a new vertex to the graph."""
```

```
        if vertex not in self.graph:
```

```
            self.graph[vertex] = []
```

```
    def add_edge(self, v1, v2):
```

```
        """Add an edge between two vertices."""
```

```
        if v1 not in self.graph:
```

```
            self.add_vertex(v1)
```

```
if v2 not in self.graph:
```

```
    self.add_vertex(v2)
```

```
self.graph[v1].append(v2)
```

```
self.graph[v2].append(v1)
```

```
def display(self):
```

```
    """Display the adjacency list of the graph."""
```

```
    for vertex in self.graph:
```

```
        print(vertex, "->", self.graph[vertex])
```

```
# Example usage:
```

```
if __name__ == "__main__":
```

```
    g = Graph()
```

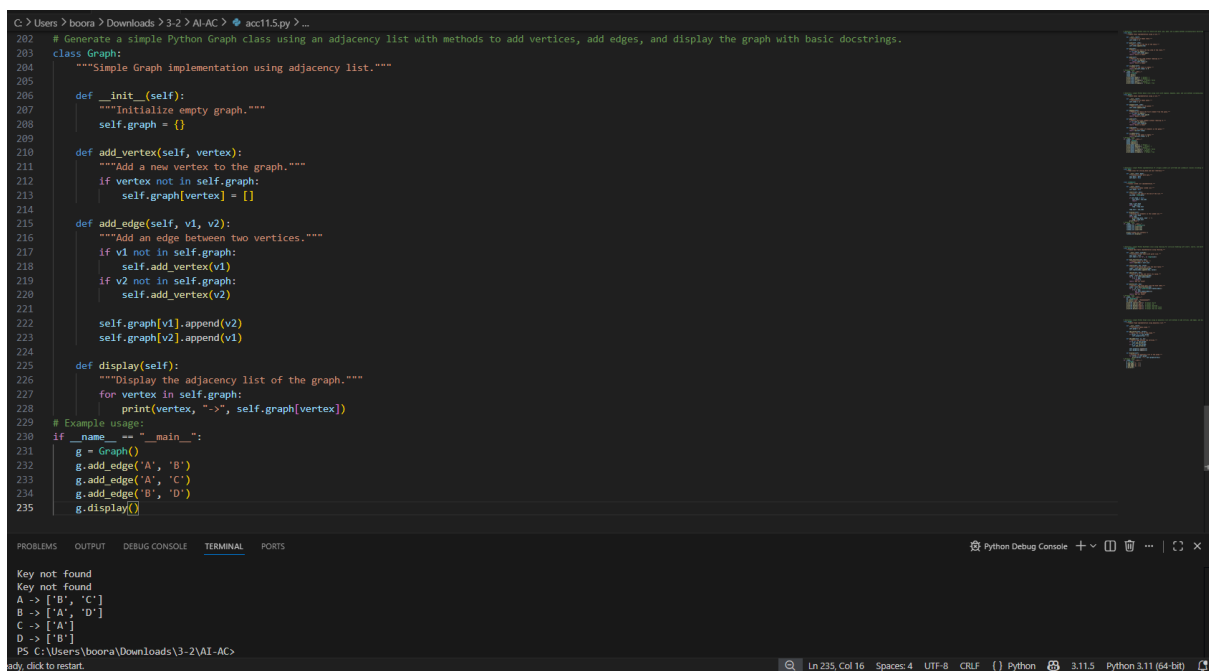
```
    g.add_edge('A', 'B')
```

```
    g.add_edge('A', 'C')
```

```
    g.add_edge('B', 'D')
```

```
    g.display()
```

Output:



```
C:\Users\boora> Downloads\3-2> AI-AC> acc11.5.py ...
202 # Generate a simple Python Graph class using an adjacency list with methods to add vertices, add edges, and display the graph with basic docstrings.
203 class Graph:
204     """Simple Graph implementation using adjacency list."""
205
206     def __init__(self):
207         """Initialize empty graph."""
208         self.graph = {}
209
210     def add_vertex(self, vertex):
211         """Add a new vertex to the graph."""
212         if vertex not in self.graph:
213             self.graph[vertex] = []
214
215     def add_edge(self, v1, v2):
216         """Add an edge between two vertices."""
217         if v1 not in self.graph:
218             self.add_vertex(v1)
219         if v2 not in self.graph:
220             self.add_vertex(v2)
221
222         self.graph[v1].append(v2)
223         self.graph[v2].append(v1)
224
225     def display(self):
226         """Display the adjacency list of the graph."""
227         for vertex in self.graph:
228             print(vertex, "->", self.graph[vertex])
229
230 # Example usage:
231 if __name__ == "__main__":
232     g = Graph()
233     g.add_edge('A', 'B')
234     g.add_edge('A', 'C')
235     g.add_edge('B', 'D')
236     g.display()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

```
Key not found
Key not found
A -> ['B', 'C']
B -> ['A', 'D']
C -> ['A']
D -> ['B']
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Ln 235, Col 16 Spaces: 4 UTF-8 CRLF Python 3.11.5 Python 3.11 (64-bit)

Task Description #6: Smart Hospital Management System – Data

Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.

- A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```
class PatientQueue:

    """Queue implementation for managing patient check-in using FIFO principle."""

    def __init__(self):

        """Initialize an empty queue."""

        self.queue = []

    def check_in(self, patient):

        """Add a patient to the queue."""

        self.queue.append(patient)

        print(patient, "checked in.")

    def treat_patient(self):

        """Remove and treat the first patient in the queue."""

        if self.is_empty():

            print("No patients waiting.")

        else:

            patient = self.queue.pop(0)

            print(patient, "is being treated.")

    def display_queue(self):

        """Display all waiting patients."""

        if self.is_empty():

            print("No patients in queue.")
```

```
else:  
    print("Waiting patients:", self.queue)
```

```
def is_empty(self):  
    """Check if queue is empty."""  
    return len(self.queue) == 0
```

Sample Usage

```
pq = PatientQueue()
```

```
pq.check_in("Patient A")
```

```
pq.check_in("Patient B")
```

```
pq.check_in("Patient C")
```

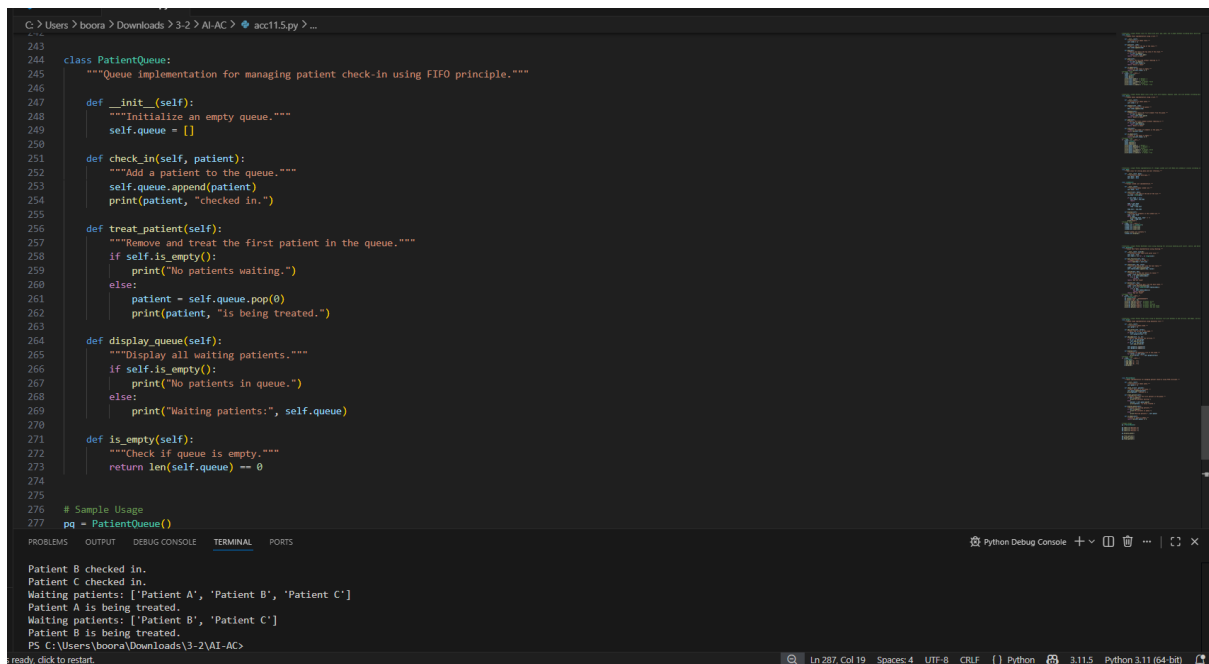
```
pq.display_queue()
```

```
pq.treat_patient()
```

```
pq.display_queue()
```

```
pq.treat_patient()
```

Output:



```
243
244 class PatientQueue:
245     """Queue implementation for managing patient check-in using FIFO principle."""
246
247     def __init__(self):
248         """Initialize an empty queue."""
249         self.queue = []
250
251     def check_in(self, patient):
252         """Add a patient to the queue."""
253         self.queue.append(patient)
254         print(patient, "checked in.")
255
256     def treat_patient(self):
257         """Remove and treat the first patient in the queue."""
258         if self.is_empty():
259             print("No patients waiting.")
260         else:
261             patient = self.queue.pop(0)
262             print(patient, "is being treated.")
263
264     def display_queue(self):
265         """Display all waiting patients."""
266         if self.is_empty():
267             print("No patients in queue.")
268         else:
269             print("Waiting patients:", self.queue)
270
271     def is_empty(self):
272         """Check if queue is empty."""
273         return len(self.queue) == 0
274
275 # Sample Usage
276 pq = PatientQueue()
277
Patient B checked in.
Patient C checked in.
Waiting patients: ['Patient A', 'Patient B', 'Patient C']
Patient A is being treated.
Waiting patients: ['Patient B', 'Patient C']
Patient B is being treated.
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Task Description #7: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from

the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List

- o Binary Search Tree (BST)

- o Graph

- o Hash Table

- o Deque

- Justify your choice in 2–3 sentences per feature.

- Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.

- A functional Python program implementing the chosen feature

with comments and docstrings. Task Description #7: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details.

4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from the list below:

- o Stack

- o Queue

- o Priority Queue

- o Linked List

- o Binary Search Tree (BST)

- o Graph

o Hash Table

o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```
class TrafficQueue:
```

```
    """Queue implementation for vehicles waiting at a traffic signal."""
```

```
    def __init__(self):
```

```
        """Initialize an empty traffic queue."""
```

```
        self.queue = []
```

```
    def add_vehicle(self, vehicle):
```

```
        """Add a vehicle to the queue."""
```

```
        self.queue.append(vehicle)
```

```
        print(vehicle, "arrived at signal")
```

```
    def pass_signal(self):
```

```
        """Allow the first vehicle to pass the signal."""
```

```
        if self.is_empty():
```

```
            print("No vehicles waiting")
```

```
        else:
```

```
vehicle = self.queue.pop(0)

print(vehicle, "passed the signal")
```

```
def display_queue(self):

    """Display vehicles currently waiting."""

    print("Vehicles waiting:", self.queue)
```

```
def is_empty(self):

    """Check if queue is empty."""

    return len(self.queue) == 0
```

Sample Usage

```
tq = TrafficQueue()
```

```
tq.add_vehicle("Car")
```

```
tq.add_vehicle("Bike")
```

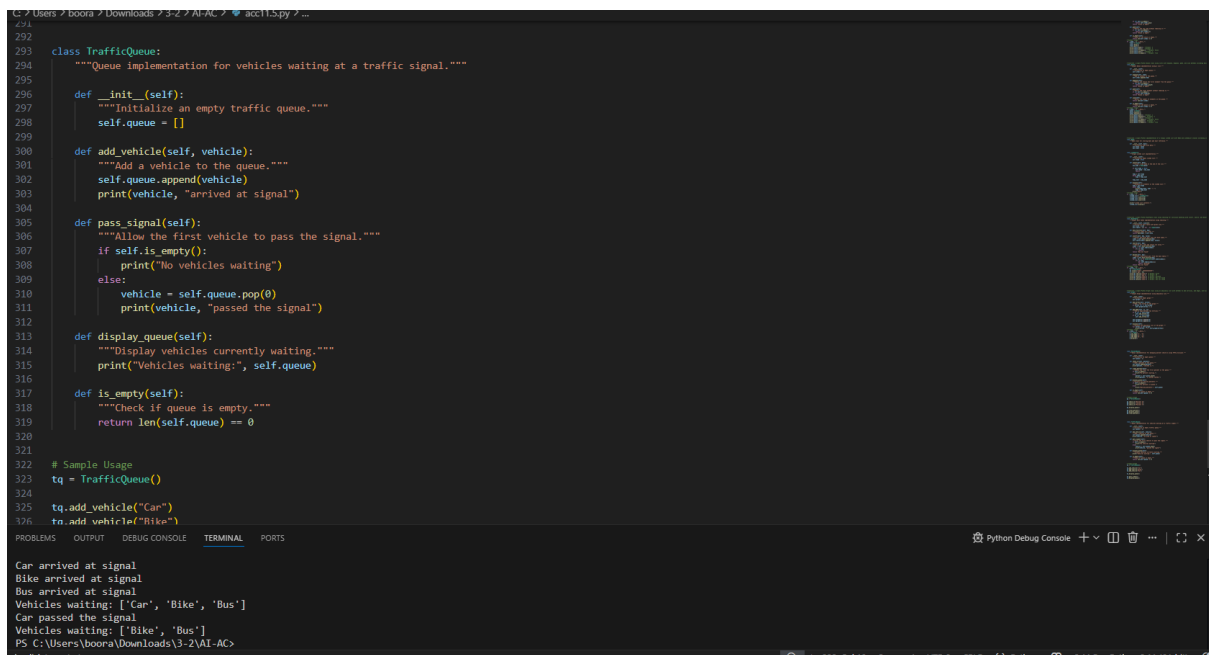
```
tq.add_vehicle("Bus")
```

```
tq.display_queue()
```

```
tq.pass_signal()
```

```
tq.display_queue()
```

Output:



```
293 class TrafficQueue:
294     """Queue implementation for vehicles waiting at a traffic signal."""
295
296     def __init__(self):
297         """Initialize an empty traffic queue."""
298         self.queue = []
299
300     def add_vehicle(self, vehicle):
301         """Add a vehicle to the queue."""
302         self.queue.append(vehicle)
303         print(vehicle, "arrived at signal")
304
305     def pass_signal(self):
306         """Allow the first vehicle to pass the signal."""
307         if self.is_empty():
308             print("No vehicles waiting")
309         else:
310             vehicle = self.queue.pop(0)
311             print(vehicle, "passed the signal")
312
313     def display_queue(self):
314         """Display vehicles currently waiting."""
315         print("Vehicles waiting:", self.queue)
316
317     def is_empty(self):
318         """Check if queue is empty."""
319         return len(self.queue) == 0
320
321 # Sample Usage
322 tq = TrafficQueue()
323
324 tq.add_vehicle("Car")
325 tq.add_vehicle("Bike")
326
327 Car arrived at signal
328 Bike arrived at signal
329 Bus arrived at signal
330 Vehicles waiting: ['Car', 'Bike', 'Bus']
331 Car passed the signal
332 Vehicles waiting: ['Bike', 'Bus']
333 PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Task Description #8: Smart E-Commerce Platform – Data Structure

Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- For each feature, select the most appropriate data structure from the list below:

o Stack

- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```
class OrderQueue:
```

```
    """Queue implementation for processing orders in FIFO order."""
```

```
    def __init__(self):
```

```
        """Initialize an empty order queue."""
```

```
        self.orders = []
```

```
    def place_order(self, order):
```

```
        """Add a new order to the queue."""
```

```
        self.orders.append(order)
```

```
        print(order, "order placed")
```

```
def process_order(self):  
    """Process the first order in the queue."""  
    if self.is_empty():  
        print("No orders to process")  
    else:  
        order = self.orders.pop(0)  
        print(order, "order processed")
```

```
def display_orders(self):  
    """Display all pending orders."""  
    print("Pending orders:", self.orders)
```

```
def is_empty(self):  
    """Check if the queue is empty."""  
    return len(self.orders) == 0
```

Sample Usage

```
oq = OrderQueue()
```

```
oq.place_order("Order101")
```

```
oq.place_order("Order102")
```

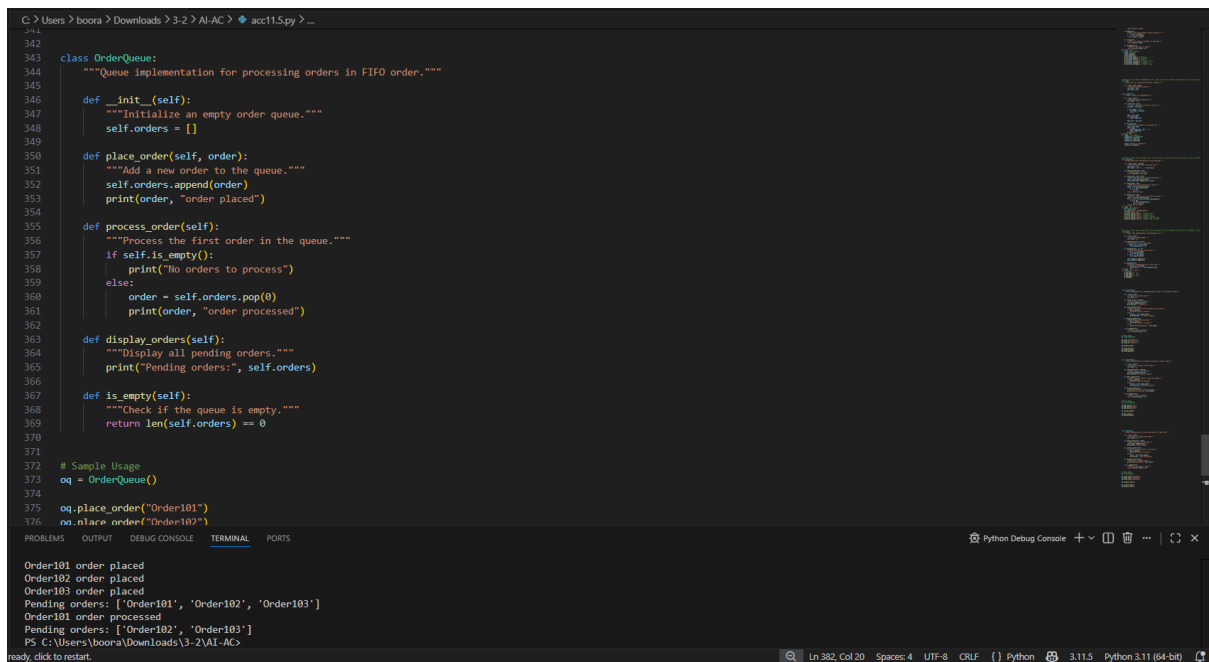
```
oq.place_order("Order103")
```

```
oq.display_orders()
```

```
oq.process_order()
```

```
oq.display_orders()
```

Output:



```
342
343 class OrderQueue:
344     """Queue implementation for processing orders in FIFO order."""
345
346     def __init__(self):
347         """Initialize an empty order queue."""
348         self.orders = []
349
350     def place_order(self, order):
351         """Add a new order to the queue."""
352         self.orders.append(order)
353         print(order, "order placed")
354
355     def process_order(self):
356         """Process the first order in the queue."""
357         if self.is_empty():
358             print("No orders to process")
359         else:
360             order = self.orders.pop(0)
361             print(order, "order processed")
362
363     def display_orders(self):
364         """Display all pending orders."""
365         print("Pending orders:", self.orders)
366
367     def is_empty(self):
368         """Check if the queue is empty."""
369         return len(self.orders) == 0
370
371
372 # Sample Usage
373 oq = OrderQueue()
374
375 oq.place_order("Order101")
376 oq.place_order("Order102")
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

Order101 order placed
Order102 order placed
Order103 order placed
Pending orders: ['Order101', 'Order102', 'Order103']
Order101 order processed
Pending orders: ['Order102', 'Order103']
PS C:\Users\boora\Downloads\3-2\AI-AC>

Justifications:

Task 1 – Stack Implementation (Justification)

Stack follows **LIFO (Last In First Out)** principle.

The last inserted element is removed first.

It is useful for operations like undo, recursion, and expression evaluation.

Task 2 – Queue Implementation (Justification)

Queue follows **FIFO (First In First Out)** principle.

Elements are processed in the same order they arrive.

It is commonly used in scheduling and task management systems.

Task 3 – Linked List (Justification)

Linked List stores elements using nodes connected by links.

It allows **dynamic memory allocation**.

Insertion and deletion operations are efficient compared to arrays.

Task 4 – Hash Table (Justification)

Hash Table stores data using **key-value pairs**.

It uses a hash function to quickly locate data.

Searching, insertion, and deletion operations are very fast.

Task 5 – Graph Representation (Justification)

Graph represents relationships between objects.

Nodes represent entities and edges represent connections.

It is useful for networks like roads, social networks, and maps.

Task 6 – Smart Hospital Management System (Justification)

Different hospital operations require different data structures.

Queue manages patient arrival order, while Priority Queue handles emergency cases.

Hash Table, BST, and Graph help manage records, appointments, and navigation efficiently.

Task 7 – Smart City Traffic Control System (Justification)

Traffic systems require organized vehicle and road management.

Queue manages vehicles at signals and Priority Queue handles emergency vehicles.

Hash Table and Graph help manage vehicle data and road network connections.

Task 8 – Smart E-Commerce Platform (Justification)

E-commerce platforms manage orders, products, and delivery routes.

Queue processes orders sequentially while Priority Queue tracks top-selling products.

Hash Table and Graph help in fast product search and delivery route planning.